Свернуть Запустить Копировать

```
Analyze the existing project structure and identify how trainable models are curre
```

LISA Flash

I cannot directly access or analyze your local file system, project structure, or specific codebase files (like `ProcessTransformer` or the experiment pipeline) as I don't have visibility into your environment.

However, based on standard machine learning project patterns and the context of adding a "recurrent gated model" to an existing pipeline involving `ProcessTransformer`, here is a summary of what you would typically need to create, assuming a modular Python-based ML framework:

Summary of Required Files and Functions

To integrate a new **Recurrent Gated Model** (e.g., GRU-LSTM hybrid or similar) without altering existing files, you would likely need to create the following components:

1. New Model Definition File

- **File:** `models/recurrent_gated_model.py` (or similar location in your models directory)
- **Function/Class Needed:**
  - A class inheriting from your base model interface (e.g., `BaseModel`, `nn.Module`, or a custom abstract class).
  - `__init__`: Define the recurrent layers (GRU/LSTM), gating mechanisms, and projection layers.
  - `forward`: The logic for processing sequential data, handling hidden states, and applying gates.
  - `get_output_shape`: Helper to determine output dimensions for downstream compatibility.

2. Data Adapter / Wrapper (If Input Format Differs)

- **File:** `adapters/gated_model_adapter.py` (Optional, depending on how strict `ProcessTransformer` is)
- **Function Needed:**
  - If the Recurrent Gated Model expects a different input tensor shape than what `ProcessTransformer` currently outputs, you may need a wrapper function to reshape or pad the data before it enters the new model.
  - If the input format matches exactly, this might not be needed.

3. Experiment Configuration Entry

- **File:** `configs/experiments.yaml` or `src/pipeline/config.py`
- **Content Needed:**
  - A new configuration block defining the hyperparameters for the recurrent gated model (e.g., `hidden_size`, `num_layers`, `dropout`, `gating_type`).

- Registration of the new model class name so the experiment runner can instantiate it dynamically.

4. Integration Point Analysis (Contextual)

Since you mentioned `ProcessTransformer`, you should verify:

- `(batch, seq_len, features)`
- `(batch, num_classes)  (batch, seq_len)`
- **Registration:** Look for a factory pattern or a registry dictionary (often in `models/__init__.py` or a `model_factory.py`) where new models are mapped by string name. You will need to add your new model class to this map.

Next Steps for You

1. **Inspect `ProcessTransformer`:** Check its `__call__` or `forward` method to see exactly what it returns (tensor shapes, types).
2. **Check the Pipeline Runner:** Find where models are instantiated (likely looking for a string-to-class mapping) to ensure you register your new model correctly.
3. **Create the Files:** Once you have the signatures from step 1, create the new model file implementing those interfaces.

Would you like me to help you draft the code for the `RecurrentGatedModel` class once you provide the signature of the `ProcessTransformer`'s output or the base model interface you are using?